
Final Report: Safety Verification of Chiron IR using Constrained Horn Clauses

Project Overview

Program verification provides mathematical guarantees that software behaves correctly, unlike testing, which can find bugs but never prove their absence. While the Chiron framework offers support for dynamic analysis techniques such as fuzzing, symbolic execution, and spectrum-based fault localization, it currently lacks the capability to *prove* that a program satisfies a given specification for *all* possible inputs.

This project bridges the gap between testing and verification in Chiron by implementing a PDR-based safety verification engine. We target properties natural to turtle graphics programs, such as spatial bounds on the turtle's position and invariants over program variables.

Team Information

Roll No.	Roll No.	Email ID
Aditi Khandelia	220061	aditikh22@iitk.ac.in
Arush Upadhyaya	220213	arushu22@iitk.ac.in
Kushagra Srivastava	220573	skushagra22@iitk.ac.in

Source Code

The complete implementation is hosted at: <https://github.com/chiron-verification/milestone-2>

- **Milestone-1 commit:** 0f99ade6d4aa2516f9e47e013ad06d29e8b8a2f7
- **Milestone-2 commit:** 4d3e3e6499b5eea03d25f0b0a3e1bf7a4dcd998b
- **Final Commit:** <TBD>

Core source files and responsibilities:

- **safety_properties.py:** Main verification API; parses user properties, applies checks/hints, and reports PASSED/FAILED/UNKNOWN with timing, and now refactored from an interactive CLI to callable `CHC_Verification(...)` with structured `ReturnValue`, timeout handling, property-scope control, and heading-grid pre-check/hint orchestration.
- **step_rules.py:** Encodes IR instructions into CHC transition rules used by the fixedpoint engine, with replaced trig bucket bounds with exact 15-degree trig encoding, introduced `BadHeading` rule generation, and added optimization-aware loop summarization and turn-safety shortcuts.
- **init_fixed_point.py:** Initializes solver state and base relations for default, specific, and universal modes, and now registers `BadHeading`, constrains universal initialization to heading-grid and integer user variables
- **z3_fixed_point.py:** Defines invariant/state-vector relations and mode-aware predicate structure and introduces the dedicated `BadHeading` relation in the same state signature as `Inv`.
- **variable_name_detection_in_IR.py:** Extracts variables from IR for building constraints and queries, and now introduces the shared `OptimizationLevel` enum and updates parser signatures/import path setup to align with the standalone `CHC-Verification` package layout.
- **heading_grid.py:** Encodes heading-grid checks to handle 15-degree alignment assumptions explicitly, and now added as a dedicated helper module exposing `heading_on_grid(h)` for reuse across initialization, transition encoding, and API-level assumptions.

- `optimization_helpers.py`: Provides static-analysis helpers for optimization decisions and loop simplification metadata, introduced to implement turn-safety dataflow, repeat-loop pattern detection, and summarizability checks used by `OptimizationLevel.BASIC`.

Current Progress

Tests

Correctness Tests

The correctness suite resides in `CHC-Verification/correctness_tests/` and is executed with `pytest` over `unittest.TestCase`-based test classes. The fixture set includes 64 `.tl` programs across the three modes, covering a wide range of semantic properties and edge cases. These fixture programs can be found in `CHC-Verification/correctness_tests/programs/`.

Evaluation modules and responsibilities

File	Role in evaluation
<code>test_default.py</code>	Validates concrete-start-state verification in default mode, including arithmetic, geometric, pen-state, directional, heading-grid, trigonometric, and nested-loop behaviors.
<code>test_specific.py</code>	Validates parameterized initialization in specific mode, with boundary-sensitive outcomes under user-supplied parameter dictionaries.
<code>test_universal.py</code>	Validates symbolic/unconstrained initialization in universal mode, API-level validation errors, heading-grid handling, and terminating-scope properties.
<code>helpers.py</code>	Shared harness for loading programs, invoking <code>CHC_Verification</code> , building assertion contexts, and standardizing property checks.
<code>README.md</code>	Documents suite structure, mode coverage, fixture list, and execution commands.

Coverage snapshot

- Total collected correctness tests: **206**
- Mode split: **46** (default), **70** (specific), **90** (universal)
- Fixture programs in `programs/`: **64** `.tl` files
- Explicitly skipped tests: **6** timeout-marked heading/trig-heavy scenarios

Category coverage across modes

- **Default mode**: arithmetic invariants, geometric bounds, pen-state properties, directional/heading checks, and advanced nested control-flow fixtures.
- **Specific mode**: same semantic families under concrete parameter assignments, including multi-parameter and chained-update programs.
- **Universal mode**: unconstrained initial-state reasoning, relational arithmetic properties, API-validation behavior, and terminating-scope checks.

How to run

From repository root (recommended):

```
cd Chiron-Framework/ChironCore
source .venv/bin/activate
cd ../../CHC-Verification/correctness_tests
pytest -q
```

Useful variants:

```

# verbose mode
pytest -v

# collect-only sanity check
pytest --collect-only -q

# run one mode file
pytest test_default.py -q

# run one test class
pytest test_specific.py::TestSpecificArithmetic -q

# run one test case
pytest test_universal.py::TestUniversalAPI::test_invalid_mode_error -q

```

Performance Tests

The CHC verification pipeline is benchmarked to stress-test solver scalability under various optimization levels. The test suite resides in *Folder: CHC-Verification/performance_tests* and is executed with `pytest` using `pytest-xdist` for parallel execution. The fixture contains 21 `.tl` programs across the three modes and cover cases that can blow up the number of solver constraints. The fixture programmes can be found in *Folder: CHC-Verification/performance_tests/programs/*.

Test Infrastructure

The performance test suite is built on Python’s `unittest` framework and reuses the shared helper infrastructure from Milestone 1. Each test case runs the verification pipeline across four configurations: two optimization levels (NONE and BASIC) combined with two hint modes (no hint, with hint).

File	Purpose
tests/test_default.py	Performance benchmarks in default mode, testing loop scaling, nesting depth, wide state, branching density, trigonometric motion, and property complexity.
tests/test_universal.py	Performance benchmarks in universal mode with unconstrained initial states.
tests/test_specific.py	Performance benchmarks in specific mode using explicit parameter assignments.
tests/helpers.py	Extended test infrastructure supporting timing measurements across all six configurations and CSV output generation.
perf_results_*.csv	Raw timing data output files containing build times, solve times, and verdicts for each test case.

Benchmark Programs

The performance suite includes 21 benchmark programs organized into six categories:

Program file	Purpose in evaluation
<i>Loop Scaling (testing solver performance as loop iterations increase)</i>	
perf_repeat_10.tl	Simple accumulator loop with 10 iterations; baseline for loop scaling.
perf_repeat_50.tl	Same accumulator with 50 iterations; tests medium-scale loop invariants.
perf_repeat_100.tl	Same accumulator with 100 iterations; tests large-scale loop invariants.
<i>Nesting Depth (testing nested loop invariant synthesis)</i>	
perf_deep_nest_3.tl	Three-level nested loops ($4^3 = 64$ total iterations); produces chained counters a, b, c .

<code>perf_deep_nest_4.tl</code>	Four-level nested loops ($3^4 = 81$ total iterations); adds counter d for deeper invariants.
<i>Wide State (testing high-dimensional state tracking)</i>	
<code>perf_wide_vars.tl</code>	Eight chained variables updated per iteration; tests 8-dimensional arithmetic invariants.
<i>Branching Density (testing branching CFG complexity)</i>	
<code>perf_many_branches.tl</code>	Six-iteration loop with four sequential if/else blocks per iteration; dense branching.
<i>Trigonometric Motion (testing heading-dependent position updates)</i>	
<code>perf_trig_10.tl</code>	<code>forward 10; right 90</code> repeated 10 times; tests trigonometric constraint scaling.
<code>perf_trig_20.tl</code>	Same motion pattern repeated 20 times; harder trigonometric reasoning.
<i>Heading/Turns (testing heading normalization cost)</i>	
<code>perf_turns_20.tl</code>	<code>right 345</code> repeated 20 times; heading visits only $\{0, 345\}$.
<code>perf_turns_50.tl</code>	Same turn pattern repeated 50 times; tests grid-check scaling.

Aggregate Results by Mode and Configuration

Tables 4, 5, and 6 present aggregate results for each verification mode across all six configurations.

Table 4: Aggregate results: **Default** mode (39 test cases).

Config	Passed	Failed	Skipped	Completion	Avg Build (s)	Avg Solve (s)
NONE	15	0	24	38.5%	0.141	2.74
BASIC	23	0	16	59.0%	0.100	2.16
NONE+H	23	0	16	59.0%	0.122	2.23
BASIC+H	23	0	16	59.0%	0.094	2.21

Table 5: Aggregate results: **Universal** mode (38 test cases).

Config	Passed	Failed	Skipped	Completion	Avg Build (s)	Avg Solve (s)
NONE	26	0	12	68.4%	0.206	4.51
BASIC	34	0	4	89.5%	0.863	4.15
NONE+H	30	0	8	78.9%	0.172	1.76
BASIC+H	36	0	2	94.7%	0.715	4.49

Table 6: Aggregate results: **Specific** mode (52 test cases).

Config	Passed	Failed	Skipped	Completion	Avg Build (s)	Avg Solve (s)
NONE	13	0	39	25.0%	0.312	1.09
BASIC	18	0	34	34.6%	0.262	1.19
NONE+H	20	0	32	38.5%	0.281	1.74
BASIC+H	19	0	33	36.5%	0.246	2.09

Figure 1 provides a visual comparison of verdict distributions across all modes and configurations.

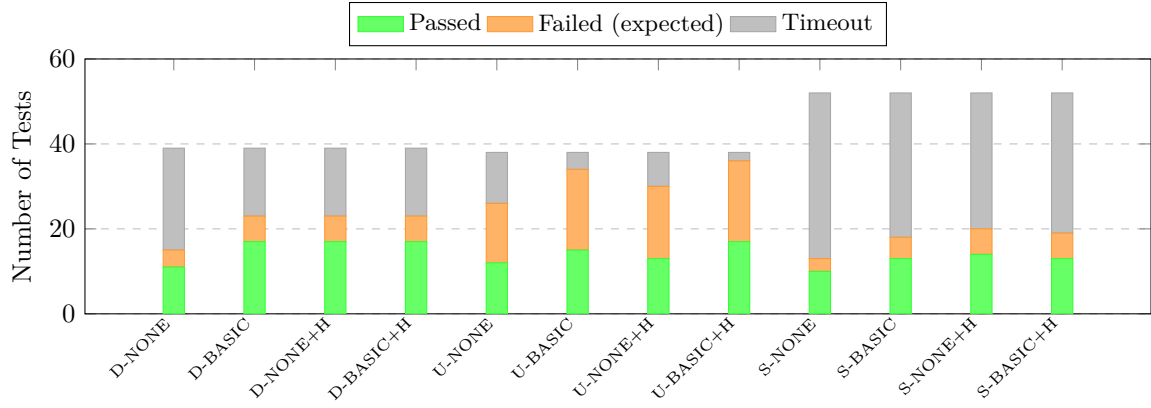


Figure 1: Verdict distribution across all modes (D=Default, U=Universal, S=Specific) and configurations. Green indicates proven properties, orange indicates expected counterexamples found, gray indicates solver timeout.

Timing Results

Tables present in *Files: CHC-Verification/performance_tests/perf_results_{default, specific, universal}.csv* present detailed timing results for each mode in all 4 configurations.

Build Time Analysis Across Verification Modes

Figure 2 shows average build times across the three verification modes for each configuration.

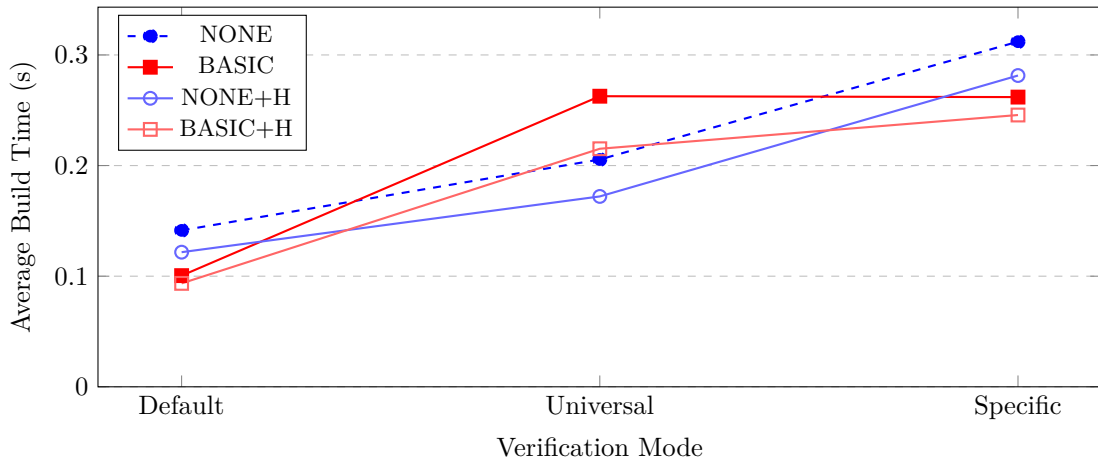


Figure 2: Average build times across verification modes. Default mode shows fastest build times; Specific mode has highest due to parameter processing overhead.

Solve Time Analysis Across Verification Modes

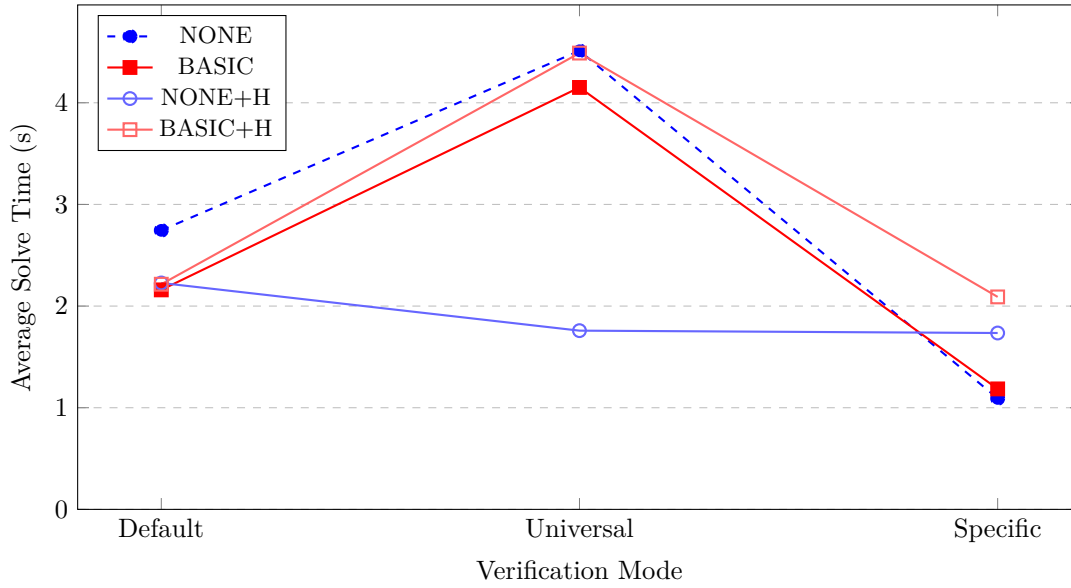


Figure 3: Average solve times across verification modes. Universal mode exhibits highest solve times due to unconstrained initial states.

Optimization Benefits

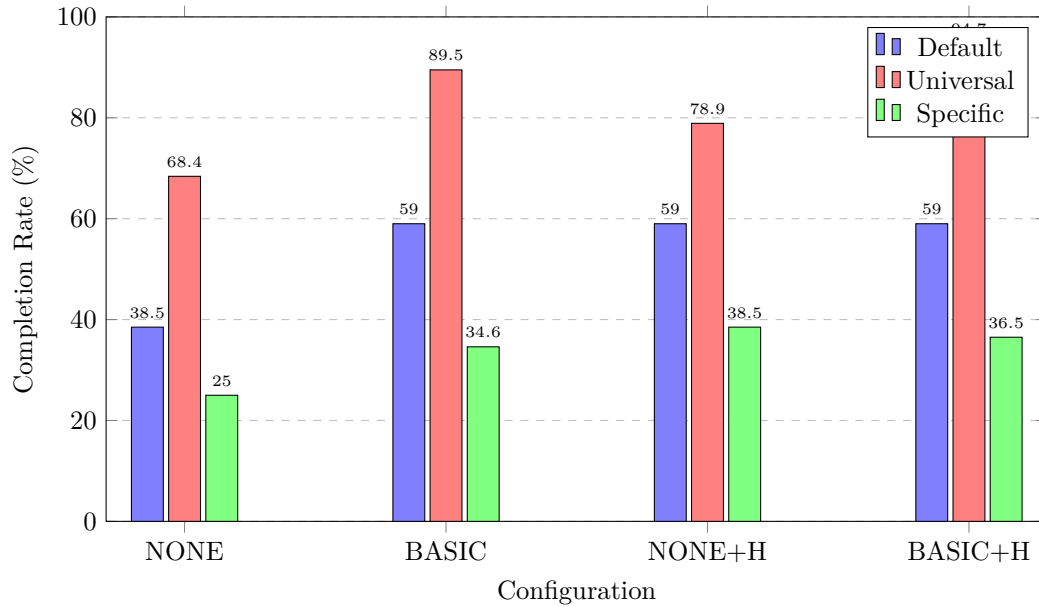


Figure 4: Test completion rates by configuration. **BASIC+H** achieves 94.7% completion in Universal mode—the highest across all mode-configuration combinations.

- **BASIC provides consistent improvement.** In Default mode, completion jumps from 38.5% (NONE) to 59.0% (BASIC); in Universal mode, from 68.4% to 89.5%.
- **Mode-dependent behavior.** Universal mode shows the largest improvement with BASIC (68.4% → 89.5%), while Specific mode shows minimal benefit (25.0% → 34.6%).

Timeout Reduction on using Hints

Table 7: Timeout reduction rates with heading hints enabled.

Mode	NONE $\rightarrow$ NONE+H	BASIC $\rightarrow$ BASIC+H
Default	33.3% (24 $\rightarrow$ 16)	0.0% (16 $\rightarrow$ 16)
Universal	33.3% (12 $\rightarrow$ 8)	50.0% (4 $\rightarrow$ 2)
Specific	17.9% (39 $\rightarrow$ 32)	2.9% (34 $\rightarrow$ 33)

Tests Uniquely Solved by Hints

Table 8: Number of tests uniquely solved when hints are enabled (timeout $\rightarrow$ success).

Mode	NONE	BASIC
Default	8	1
Universal	5	2
Specific	7	2
Total	20	5

- **Combining BASIC with HINTS is optimal.** It achieves the highest completion rates (59.0% Default, 94.7% Universal, 36.5% Specific).
- **Enable hints for trigonometric programs.** When verifying programs with heading-dependent motion (forward/backward), hints provide a significant timeout reduction.

Overall Performance

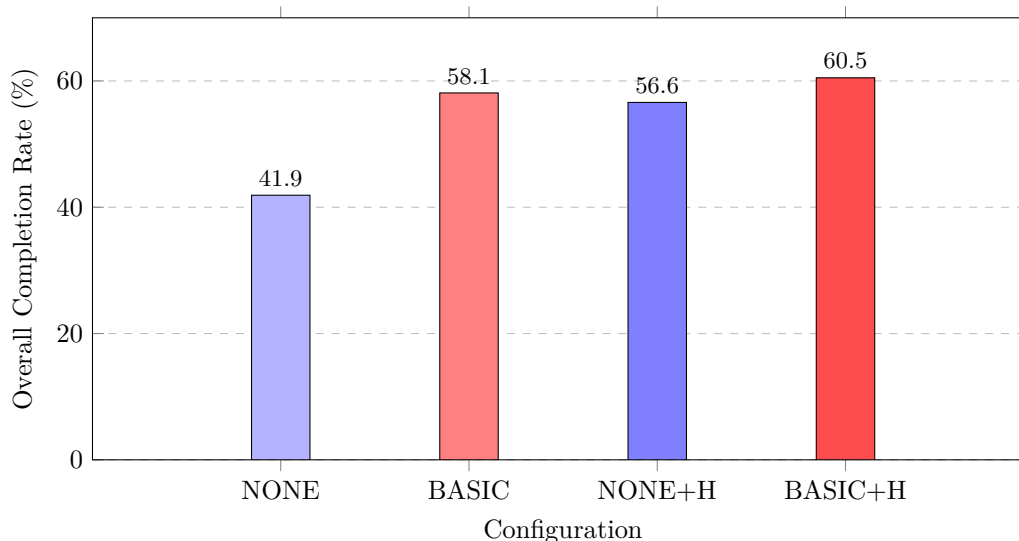


Figure 5: Overall completion rates across all 129 tests. BASIC+H achieves the highest overall rate (60.5%).

Running the Performance Tests

```
cd Chiron-Framework/ChironCore/CHC_Verification/performance_tests
pytest test_default.py -v      # Default mode benchmarks
pytest test_universal.py -v    # Universal mode benchmarks
pytest test_specific.py -v     # Specific mode benchmarks
```

Results are written to `perf_results_<mode>.csv` for analysis.

Analysis

Strengths of the implementation

Limitations of the implementation

Future Plans

AI Use in the Project

In this project, we utilized AI tools to enhance our workflow and improve the quality of our work.

1. **Are you using AI for coding your implementation?** Yes, we used AI tools in this project.
2. **If yes, what models are you using?** We used models like OpenAI's GPT-5.2 Codex and GitHub Copilot.
3. **If using AI, how are you using AI and what percentage (approx) of your code is AI generated?** We used AI primarily for generating boilerplate code, suggesting improvements, and debugging. Approximately 40% – 50% of our code was generated or assisted by AI tools.
 - AI was primarily used for generating boilerplate code and usage of correct syntax, which was then reviewed and modified by us to fit our specific requirements.
 - We also used AI for testing, where it helped in generating test cases and suggesting improvements to our existing tests.
 - Additionally, AI was used for the generation of documentation and figures in the report, where it assisted in summarizing the directory structure and usage guide, as well as making the figures in the report with careful and structured instructions about the layout.

Declaration

- We declare that the work presented in this report is our own and has been carried out in accordance with the guidelines set out by the course.
- We have not copied any part of this report from any other source, and we have not submitted this report to any other course or institution for assessment.
- We understand that any violation of this declaration may result in disciplinary action.